Shared by Aakash. This is a copy of a chat between Claude and Aakash. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden when shared

I'm doing a lab where I fix and extend a small Pygame "Catch the Falling Objects" game. I'll work with you one task at a time. Here's the project README for context, followed by all the source files.

README summary: A basket moves left/right at the bottom; objects spawn at the top and fall; score for catches, miss counter, game over after too many misses. Python 3.10+, Pygame, controls are left/right arrows.

Please read the code and give me a short summary of how it's structured (what each file does and how the game loop flows). Don't change anything yet.

Here's how the project is laid out and how it runs. I've only read the code, and nothing has been changed.

What each file does

- **main.py** is the entry point. It initializes Pygame, opens a 700x500 window, and runs the main loop at 60 FPS. It contains no game logic and only forwards events and key state to the engine.
- **game_engine.py** holds `GameEngine`, which owns the game state: the basket, the list of falling objects, the score, the miss count, and the `game_over` flag. It handles input, spawning, updating, catch and miss detection, and restarting.
- **basket.py** defines `Basket`, a plain data class with position, size, and speed. It also has some unused speed-boost fields (`boosted_frames`, `normal_speed`, `boost_speed`) that look like scaffolding for Task 4. `get_rect()` converts the center-based position into a `pygame.Rect`.
- **falling_object.py** defines `FallingObject`, a circle with position, radius, speed, and color. `update()` moves it down each frame, and `is_past_bottom()` checks whether it has left the screen.
- **collision.py** has a single function, `is_caught(basket_rect, obj)`, that decides whether an object counts as caught.

- **renderer.py** contains all the drawing code (scene, text, and the game-over banner) and the screen constants `WIDTH`, `HEIGHT`, and `WINDOW_SIZE`. It's kept separate from the logic.

Game loop flow

Each frame in `main.py` does the following:

1. Process events. Quit stops the loop, and a `KEYDOWN` goes to `engine.handle_keydown()`, where `R` restarts only if the game is over.
2. Read held keys with `pygame.key.get_pressed()` and pass them to `engine.handle_input()`, which moves the basket left or right and clamps it to the screen.
3. Call `engine.update()`, which does this in order:
   - Counts down the spawn timer and spawns a new object every 50 frames at a random x.
   - Moves every object down.
   - Checks for catches, adding to the score and removing caught objects.
   - Removes objects that fell past the bottom, adds them to `misses`, and sets `game_over` once misses reach 5.
4. Call `engine.draw()`, which delegates to the renderer for the scene, score, misses, and the game-over banner.
5. Flip the display.

When the game is over, both `handle_input` and `update` return early, so the game freezes until `R` is pressed.

Things I noticed

These line up with the tasks mentioned in the starter comments:

- **Catch loop bug:** `update()` removes items from `self.objects` while iterating over it, which can skip objects.
- **Catch check bug:** `is_caught` only compares the object's x to the basket's horizontal span, ignoring y. Any object at the right x counts as caught, regardless of how high up it is.
- **Boundary handling:** The basket is clamped by its center to `0..WIDTH`, so half of it can hang off either edge.
- **Boost scaffolding:** The unused boost fields in `Basket` and the lack of any boost logic in the engine match the "build it from scratch" note for Task 4.

Which task would you like to start with?

Let's start with Task 1: Fix the falling-object collision bug. There are two problems, which match what you noticed:

1. is_caught in game/collision.py only checks whether the object's x position is within the basket's left/right edges. It never checks height, so an object scores the instant it spawns at the top if the basket happens to be under its x position.
2. The catch loop in update() in game/game_engine.py removes objects from the same list it is iterating over, which can silently skip an object when two are adjacent.

Expected behavior: an object is caught only when it has actually reached basket height, and every catchable object is checked every frame, so two adjacent catchable objects are both credited.

Please fix both. Give me the full updated code for the changed functions (or files), and briefly explain what was wrong and what you changed. Keep the existing code style, and don't change anything outside Task 1.

Show more

What was wrong

1. `is_caught` **ignored height.** It only compared the object's x to the basket's left and right edges, so an object counted as caught the moment it spawned above a basket sitting under that x.
2. **The catch loop mutated the list it was iterating over.** Calling `self.objects.remove(obj)` inside `for obj in self.objects` shifts the later items down by one, so the loop skips the item right after each removal. Two adjacent catchable objects would only credit one.

Fixes

game/collision.py

python

```
"""
collision: figures out whether a falling object is within the basket.
"""


def is_caught(basket_rect, obj):
    # Horizontal: the object's center is within the basket's span.
    in_x = basket_rect.left <= obj.x <= basket_rect.right
    # Vertical: the circle overlaps the basket's height band. The first
    # check means it has reached the basket's top; the second means it
    # hasn't already fallen completely past the bottom, so a basket
    # sliding under an object that already missed can't catch it.
    in_y = (obj.y + obj.radius >= basket_rect.top
            and obj.y - obj.radius <= basket_rect.bottom)
    return in_x and in_y
```

game/game_engine.py: replace the catch loop inside `update()`:

python

```
        basket_rect = self.basket.get_rect()
        caught = [o for o in self.objects if is_caught(basket_rect, o)]
        if caught:
            self.objects = [o for o in self.objects if o not in caught]
            self.score += len(caught)
```

Everything else in `update()` stays the same. The spawn timer, the object updates, and the miss handling below the catch check are untouched.

What changed

- `is_caught` now requires the object to be at basket height as well as over the basket's x-range. I used the circle's edge (`y + radius`) rather than its center, so it counts as caught when it visually touches the basket's top.
- The catch check now builds a `caught` list in one pass without modifying anything, then rebuilds `self.objects` without those items. This is the same pattern the miss handling already uses just below it, so every object is checked every frame and each catch is credited.

One design choice to flag: I added the upper bound on the vertical check (`obj.y - obj.radius <= basket_rect.bottom`), which goes slightly beyond what you described. Without it, an object that already fell past the basket's top while you were off to the side could still be "caught"

if you slid underneath it afterward. If you'd rather keep the simpler "reached basket height" rule only, drop the second condition.

To test, hold the basket in one spot and watch that objects score only when they touch it, not on spawn. Then try to line up two objects falling close together and confirm the score jumps by 2.

Task 1 works: objects score only when they reach the basket, and adjacent catches are both credited. Now Task 2: Improve basket control.

The basket can currently move half off either edge of the screen because the boundary check clamps its center to 0..WIDTH and ignores its width. Fix the boundary handling so the basket always stays fully on screen. Also make sure movement feels smooth and responsive when a direction key is held down continuously.

Give me the updated code for the affected files and a short explanation of the changes. Don't touch the collision logic we just fixed.

What was wrong

`handle_input` clamped the basket's center to `0..WIDTH`. The basket is positioned by its center (see `get_rect()`), so at `x = 0` half of it was off the left edge, and at `x = WIDTH` half was off the right.

The fix

Only `handle_input` in **game/game_engine.py** changes:

python

```python
    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        if keys_pressed[pygame.K_LEFT]:
            self.basket.x -= self.basket.speed
        if keys_pressed[pygame.K_RIGHT]:
            self.basket.x += self.basket.speed
        # Boundary handling: clamp the center so the whole basket stays on
        # screen, i.e. its edges stop at 0 and WIDTH.
        half_width = self.basket.width / 2
        self.basket.x = max(half_width, min(WIDTH - half_width, self.basket.x))
```

Nothing else in the file changes, and `collision.py` is untouched.

What changed

- The clamp range is now `half_width .. WIDTH - half_width` instead of `0 .. WIDTH`. With the default 90px basket, the center stops at 45 and 655, so the left edge sits exactly at 0 and the right edge exactly at 700.
- It reads `self.basket.width`, so it still works if the width changes later.

On the "smooth and responsive" part

I didn't need to change anything here, because the existing code already behaves well:

- Movement uses `pygame.key.get_pressed()`, which reports the held state every frame. It does not depend on `KEYDOWN` events, so there's no OS key-repeat delay or stutter. The basket moves the instant you press and keeps moving at a constant 5px per frame (300px/s at 60 FPS) while held.
- The clamp runs after the movement every frame, so the basket stops flush against the wall rather than overshooting and snapping back.
- Holding both arrows cancels out, which is a sensible result.

I held back from adding things like acceleration or delta-time movement, since that goes beyond what you described, and Task 4 will likely touch `basket.speed` for the boost. If you want a different feel (faster, or with acceleration), I can add it.

To test, hold Left and Right in turn and confirm the basket's edge stops exactly at each screen edge with no part hanging off, then confirm it glides without hitching along the way.

Task 2 works: the basket stays fully on screen. Now Task 3: Implement controlled object spawning.

Improve the spawner so that:
- objects appear at varied horizontal positions and at varied (randomized) intervals, instead of a fixed 50-frame interval with a plain random x
- objects never spawn outside the playable area (account for the object's radius)
- it avoids spawning repeatedly in the same spot (e.g. enforce a minimum horizontal distance from the previous spawn)
- there is a cap on the number of objects on screen at once

Put the tunable values (min/max spawn interval, max objects on screen, min x distance between spawns) in clearly named constants at the top of the file. Give me the updated code and a short explanation. Don't change the collision or basket logic.

What was wrong

Spawning had three weaknesses. The interval was a fixed 50 frames. The x was `randint(20, WIDTH - 20)`, a hardcoded margin that didn't match the object's actual radius. And nothing remembered the previous spawn or limited how many objects could exist.

The changes

All changes are in **game/game_engine.py**. `collision.py` and `basket.py` are untouched.

Constants at the top (replace `SPAWN_INTERVAL_FRAMES`):

python

```
SPAWN_INTERVAL_MIN_FRAMES = 30   # shortest wait between spawns
SPAWN_INTERVAL_MAX_FRAMES = 70   # longest wait between spawns
MAX_OBJECTS_ON_SCREEN = 6        # no new spawns while this many are falling
MIN_SPAWN_X_DISTANCE = 120       # min horizontal gap from the previous spawn
OBJECT_RADIUS = 14
OBJECT_SPEED = 3
MAX_MISSES = 5
```

`__init__`, with one new field:

python

```
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.last_spawn_x = None
        self.score = 0
        self.misses = 0
        self.game_over = False
```

Spawning, replacing `_spawn_object` and adding a helper:

python

```
    def _pick_spawn_x(self):
        low = OBJECT_RADIUS
        high = WIDTH - OBJECT_RADIUS
        if self.last_spawn_x is None:
            return random.randint(low, high)

        # Valid zones: far enough left or far enough right of the last spawn.
        zones = []
        left_high = self.last_spawn_x - MIN_SPAWN_X_DISTANCE
        right_low = self.last_spawn_x + MIN_SPAWN_X_DISTANCE
        if left_high >= low:
            zones.append((low, left_high))
        if right_low <= high:
            zones.append((right_low, high))

        if not zones:
            # Min distance is too large to satisfy; use the farthest edge.
            return low if self.last_spawn_x > WIDTH / 2 else high

        # Weight by zone width so every valid x is equally likely.
        zone_low, zone_high = random.choices(
            zones, weights=[b - a + 1 for a, b in zones]
        )[0]
        return random.randint(zone_low, zone_high)

    def _spawn_object(self):
        x = self._pick_spawn_x()
        self.last_spawn_x = x
        self.objects.append(
            FallingObject(x=x, y=-OBJECT_RADIUS, radius=OBJECT_RADIUS, speed=OBJECT_SPE
        )
```

In `update()`, replace the spawn block only:

python

```
        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            if len(self.objects) < MAX_OBJECTS_ON_SCREEN:
                self._spawn_object()
            self.frames_until_spawn = random.randint(
                SPAWN_INTERVAL_MIN_FRAMES, SPAWN_INTERVAL_MAX_FRAMES
            )
```

The rest of `update()` stays as you have it.

What changed

- **Randomized intervals:** each spawn picks a new wait between 30 and 70 frames (about 0.5 to 1.2 seconds).
- **Inside the playable area:** x is drawn from `OBJECT_RADIUS .. WIDTH - OBJECT_RADIUS`, so a circle never pokes past a side edge. It also spawns at `y = -OBJECT_RADIUS`, just above the screen, instead of the hardcoded `-14`. The radius is now passed explicitly so the spawn bounds and the object always agree.
- **No repeated spots:** the engine remembers `last_spawn_x` and picks the next x from the zones at least `MIN_SPAWN_X_DISTANCE` to its left or right. I compute the valid zones directly instead of retrying random picks until one works, so there's no loop that could spin forever, and the result is still uniform over the allowed positions. If the distance is set so large that no zone fits, it falls back to the far edge.
- **Cap:** when the cap is hit, that spawn slot is skipped and the timer resets, so spawning picks back up as objects are caught or missed.
- **Restart:** `handle_keydown` calls `__init__()`, so `last_spawn_x` resets automatically on restart.

One behavior to be aware of: at speed 3 an object takes about 170 frames to cross the screen, so with these intervals you'll typically see 3 to 5 objects at once. The cap of 6 only kicks in during a streak of short intervals. Lower `MAX_OBJECTS_ON_SCREEN` if you want to see it clearly while testing.

To test, watch that consecutive objects are never in nearly the same column, that gaps between them vary, and that nothing is ever clipped at the left or right edge. Set `MAX_OBJECTS_ON_SCREEN = 2` temporarily to confirm the cap holds.

Task 3 works: spawning is varied, bounded, and capped. Now Task 4: Add a temporary basket speed boost.

Requirements:
- The player activates it with a key (e.g. SPACE).
- While active, the basket moves faster (use a clearly named multiplier or boost speed constant).
- Show a clear on-screen indicator while it's active, e.g. a "BOOST" label plus a shrinking timer bar, and/or a colour change on the basket.
- It lasts a fixed duration, then the basket automatically returns to normal speed.
- Add a short cooldown so it can't be spammed, and show when it's ready.
- Pressing R to restart should reset the boost state.

I noticed Basket already has unused boosted_frames, normal_speed and boost_speed fields. Please reuse those if they fit rather than adding duplicates. Put the tunable values (boost duration, cooldown, speed) in clearly named constants. Give me the updated code for basket.py, game_engine.py, and renderer.py, with a short explanation of the changes. Don't change the collision or spawning logic.

Show more

What I changed

- **basket.py** gets the boost state and logic, reusing `boosted_frames`, `normal_speed`, `boost_speed` and the existing `speed`. `speed` is what `handle_input` already reads, so movement code needed no changes. I added one field, `cooldown_frames`, because nothing existing could hold it.
- **game_engine.py** gets SPACE handling in `handle_keydown`, a per-frame `basket.update_boost()` call in `update()`, and a HUD call in `draw()`.
- **renderer.py** gets a basket colour change while boosted and a new `draw_boost_hud()` function.

Collision and spawning are untouched. Both are reproduced below exactly as they stood after Tasks 1 to 3, because you asked for the full files.

game/basket.py

python

```
"""
Basket: the player-controlled catcher at the bottom of the screen.
"""

import pygame

BOOST_SPEED_MULTIPLIER = 2.0     # boosted speed = normal speed * this
BOOST_DURATION_FRAMES = 90       # how long a boost lasts (1.5 s at 60 FPS)
BOOST_COOLDOWN_FRAMES = 150      # recharge time after a boost ends (2.5 s)


class Basket:
    def __init__(self, x, y, width=90, height=24, speed=5):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = speed
        self.boosted_frames = 0          # frames of boost remaining
        self.cooldown_frames = 0         # frames of cooldown remaining
        self.normal_speed = speed
        self.boost_speed = speed * BOOST_SPEED_MULTIPLIER

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )

    def is_boosted(self):
        return self.boosted_frames > 0

    def can_boost(self):
        return self.boosted_frames == 0 and self.cooldown_frames == 0

    def activate_boost(self):
        """Start a boost if one is available. Returns True if it started."""
        if not self.can_boost():
            return False
        self.boosted_frames = BOOST_DURATION_FRAMES
        self.speed = self.boost_speed
        return True

    def update_boost(self):
```

```
        """Call once per frame: run down the boost, then the cooldown."""
        if self.boosted_frames > 0:
            self.boosted_frames -= 1
            if self.boosted_frames == 0:
                self.speed = self.normal_speed
                self.cooldown_frames = BOOST_COOLDOWN_FRAMES
        elif self.cooldown_frames > 0:
            self.cooldown_frames -= 1

    def boost_fraction(self):
        """Fraction of the active boost left (1.0 just started, 0.0 over)."""
        return self.boosted_frames / BOOST_DURATION_FRAMES

    def cooldown_fraction(self):
        """Fraction of the cooldown left (1.0 just started, 0.0 ready)."""
        return self.cooldown_frames / BOOST_COOLDOWN_FRAMES
```

game/game_engine.py

python

```
"""
GameEngine: owns the basket and all falling objects.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_MIN_FRAMES = 30   # shortest wait between spawns
SPAWN_INTERVAL_MAX_FRAMES = 70   # longest wait between spawns
MAX_OBJECTS_ON_SCREEN = 6        # no new spawns while this many are falling
MIN_SPAWN_X_DISTANCE = 120       # min horizontal gap from the previous spawn
OBJECT_RADIUS = 14
OBJECT_SPEED = 3
MAX_MISSES = 5


class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.last_spawn_x = None
        self.score = 0
        self.misses = 0
        self.game_over = False

    def _pick_spawn_x(self):
        low = OBJECT_RADIUS
        high = WIDTH - OBJECT_RADIUS
        if self.last_spawn_x is None:
            return random.randint(low, high)

        # Valid zones: far enough left or far enough right of the last spawn.
        zones = []
        left_high = self.last_spawn_x - MIN_SPAWN_X_DISTANCE
        right_low = self.last_spawn_x + MIN_SPAWN_X_DISTANCE
        if left_high >= low:
            zones.append((low, left_high))
        if right_low <= high:
```

```
            zones.append((right_low, high))

        if not zones:
            # Min distance is too large to satisfy; use the farthest edge.
            return low if self.last_spawn_x > WIDTH / 2 else high

        # Weight by zone width so every valid x is equally likely.
        zone_low, zone_high = random.choices(
            zones, weights=[b - a + 1 for a, b in zones]
        )[0]
        return random.randint(zone_low, zone_high)

    def _spawn_object(self):
        x = self._pick_spawn_x()
        self.last_spawn_x = x
        self.objects.append(
            FallingObject(x=x, y=-OBJECT_RADIUS, radius=OBJECT_RADIUS, speed=OBJECT_SPE
        )

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        if keys_pressed[pygame.K_LEFT]:
            self.basket.x -= self.basket.speed
        if keys_pressed[pygame.K_RIGHT]:
            self.basket.x += self.basket.speed
        # Boundary handling: clamp the center so the whole basket stays on
        # screen, i.e. its edges stop at 0 and WIDTH.
        half_width = self.basket.width / 2
        self.basket.x = max(half_width, min(WIDTH - half_width, self.basket.x))

    def handle_keydown(self, key):
        if self.game_over:
            if key == pygame.K_r:
                self.__init__()      # fresh Basket too, so boost state resets
            return
        if key == pygame.K_SPACE:
            self.basket.activate_boost()

    def update(self):
        if self.game_over:
            return

        self.basket.update_boost()
```

```
        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            if len(self.objects) < MAX_OBJECTS_ON_SCREEN:
                self._spawn_object()
            self.frames_until_spawn = random.randint(
                SPAWN_INTERVAL_MIN_FRAMES, SPAWN_INTERVAL_MAX_FRAMES
            )

        for obj in self.objects:
            obj.update()

        basket_rect = self.basket.get_rect()
        caught = [o for o in self.objects if is_caught(basket_rect, o)]
        if caught:
            self.objects = [o for o in self.objects if o not in caught]
            self.score += len(caught)

        missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
        if missed:
            self.objects = [o for o in self.objects if not o.is_past_bottom(HEIGHT)]
            self.misses += len(missed)
            if self.misses >= MAX_MISSES:
                self.game_over = True

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.basket, self.objects)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
        renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 3
        renderer.draw_boost_hud(surface, font, self.basket)

        if self.game_over:
            renderer.draw_banner(surface, font, f"Game Over! Final score: {self.score}.
```

game/renderer.py

python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (25, 30, 45)
COLOR_BASKET = (150, 110, 70)
COLOR_BASKET_BOOST = (255, 200, 60)
COLOR_TEXT = (255, 255, 255)
COLOR_BOOST_ACTIVE = (255, 200, 60)
COLOR_BOOST_COOLDOWN = (130, 140, 160)
COLOR_BOOST_READY = (100, 220, 130)
COLOR_BAR_BG = (60, 66, 85)

BOOST_BAR_POS = (10, 92)
BOOST_BAR_SIZE = (160, 12)


def draw_scene(surface, basket, objects):
    surface.fill(COLOR_BG)
    for obj in objects:
        pygame.draw.circle(surface, obj.color, (int(obj.x), int(obj.y)), obj.radius)
    basket_color = COLOR_BASKET_BOOST if basket.is_boosted() else COLOR_BASKET
    pygame.draw.rect(surface, basket_color, basket.get_rect(), border_radius=6)


def draw_boost_hud(surface, font, basket):
    """Boost label plus a bar: shrinks while active, refills while recharging."""
    if basket.is_boosted():
        label, color, fill = "BOOST!", COLOR_BOOST_ACTIVE, basket.boost_fraction()
    elif basket.cooldown_frames > 0:
        label, color, fill = "Boost recharging...", COLOR_BOOST_COOLDOWN, 1 - basket.co
    else:
        label, color, fill = "Boost ready [SPACE]", COLOR_BOOST_READY, 1.0

    draw_text(surface, font, label, (10, 62), color)

    x, y = BOOST_BAR_POS
    w, h = BOOST_BAR_SIZE
    pygame.draw.rect(surface, COLOR_BAR_BG, (x, y, w, h), border_radius=4)
```

```
    fill_w = int(w * fill)
    if fill_w > 0:
        pygame.draw.rect(surface, color, (x, y, fill_w, h), border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)
```

How it works

- **Activation:** SPACE calls `basket.activate_boost()`, which only starts a boost if `can_boost()` is true (not already boosted and not on cooldown). Pressing it at other times does nothing, so it can't be spammed. It uses `KEYDOWN`, so holding SPACE doesn't re-trigger it.
- **Speed:** `activate_boost()` sets `speed = boost_speed` (5 × 2.0 = 10 px/frame). When the boost runs out, `update_boost()` sets it back to `normal_speed`, so `handle_input` needed no changes. The Task 2 clamp still applies, so the basket can't leave the screen even at double speed.
- **Duration and cooldown:** `update_boost()` runs once per frame. It counts `boosted_frames` down to 0, restores normal speed, then starts the cooldown and counts that down. The cooldown starts when the boost ends, so the player always gets the full recharge time.
- **Indicator:** while boosted, the basket turns gold, the label reads "BOOST!", and the bar shrinks. During cooldown the label is grey and the bar refills. When ready, the label turns green and reads "Boost ready [SPACE]".
- **Restart:** `handle_keydown` calls `self.__init__()`, which builds a new `Basket`, so the boost timers and speed reset along with everything else. SPACE is ignored on the game-over screen.
- **Where the constants live:** `BOOST_SPEED_MULTIPLIER`, `BOOST_DURATION_FRAMES` and `BOOST_COOLDOWN_FRAMES` are at the top of `basket.py`, not `game_engine.py`. `game_engine.py` imports from `renderer.py`, so the renderer can't import the engine's constants without a circular import. The renderer reads the basket's `boost_fraction()` and `cooldown_fraction()` instead.

To test, press SPACE and confirm the basket turns gold, moves visibly faster, and the bar shrinks over 1.5 seconds. Then confirm the label switches to "recharging", that SPACE does nothing until it turns green, and that pressing R after game over starts with "Boost ready".

